

Milestone 6 Deliverable Structure

Milestone 6 — Deployment & Documentation — is about turning your project from a research notebook into a working product with clarity, reproducibility, and accessibility.

As there is quite a lot involved here, you can begin working on the various parts of this milestone by equitably assigning tasks within the group.

The recommended structure is an ideal way to finish this with a style. However, getting the most out of this milestone is about doing what you can out of your available resources.

You'll produce three key work products:

- Deployment (live or local demo + API)
- Comprehensive Technical Documentation
- Final Project Report

1. Deployment Deliverable

Acceptable Deployment Options

- Hugging Face Spaces (for apps built with Streamlit, Gradio, or Flask)
- Streamlit/Gradio app hosted locally or on a cloud instance
- REST API deployed via Flask/FastAPI on Render, Hugging Face, or similar
- Colab or Kaggle Demo Notebook (fallback for GPU-heavy models)

Deployment Requirements

- Clear instructions to run (README.md or /deployment/README.md)
- Inputs/Outputs clearly described (with examples)
- Optional demo video or screenshots

2. Comprehensive Documentation (Core Deliverable)

This is the “how it works, what’s inside, how to reproduce” part. You can structure this as a set of Markdown files under a /docs folder, or as one detailed PDF report.

Here’s the recommended breakdown.

A. Overview

- Purpose: Briefly restate the problem and objective.
- Architecture Summary: Include your final system diagram showing data flow from input → model → output.
- Deployed Components: Describe what's live (frontend, API, backend model, etc.) and where.

B. Technical Documentation

This is the heart of Milestone 6 - it ensures future teams or evaluators can understand, reproduce, or extend your work.

Section	Description
1. Environment Setup	List dependencies, libraries, Python version, hardware requirements. Provide install instructions or a requirements.txt / environment.yml.
2. Data Pipeline	Summarize preprocessing, feature extraction, and dataset sources. Include licensing info and where data lives.
3. Model Architecture	Diagram + summary of final architecture (e.g., YOLOv8, Attention U-Net, RAG pipeline). Mention chosen hyperparameters.
4. Training Summary	Reference Milestone 4 briefly — mention training time, epochs, optimizer, loss, and key metrics achieved.
5. Evaluation Summary	Reference Milestone 5 — highlight key performance numbers and insights.
6. Inference Pipeline	Explain how data moves from input → model → output. Include code snippet for inference function or API call.
7. Deployment Details	- Platform used (HF Spaces / Render / Streamlit) - How model is hosted - How to interact with it (API endpoint or UI) - Example command or request
8. System Design Considerations	Discuss architecture design, scalability, modularity, data flow. For RAG projects — mention how DB and retriever interact.
9. Error Handling & Monitoring	Optional: mention how exceptions, latency, or failures are managed.
10. Reproducibility Checklist	Paths, seeds, checkpoints, sample config files - make it possible to replicate results.

C. User Documentation (for Non-Technical Users)

If the deployed system is user-facing:

- **App Overview:** Purpose and use cases
- **Input Description:** How users enter queries or data
- **Output Description:** What they see or hear
- **Step-by-Step Instructions:**
 - How to launch the app
 - How to interact with it
 - Example queries or images
- **Troubleshooting:** What to do if it doesn't load, or if errors occur
- **Screenshots:** Show a few example interactions

D. API Documentation (if applicable)

For projects exposing REST or inference endpoints:

Section	Example
Base URL	<code>https://huggingface.co/spaces/team/project-name</code>
POST /predict	Input format (JSON) and response example
GET /status	Optional health-check endpoint
Example Curl Request	<code>curl -X POST -H "Content-Type: application/json" -d '{"query": "Can I return a shirt?"}' <endpoint></code>
Response Format	JSON keys and meanings

E. Licensing & Dataset References

Include in `/docs/LICENSES.md`:

- Code license (e.g., MIT)
- Dataset licenses
- Model sources and citations

F. Future Work / Maintenance Notes

End with practical pointers:

- Possible extensions (new datasets, models, features)
- Known limitations (performance, coverage, hardware)
- How to retrain or update the model
- Contacts / maintainers

3. Final Project Report (for submission)

A written report in PDF, you can mirror the above structure with added narrative and visuals:

1. Title Page
2. Abstract
3. Introduction
4. Literature Review (Milestone 1)
5. Dataset and Methodology (Milestone 2–3)
6. Model Development and Hyperparameter Tuning (Milestone 4)
7. Evaluation & Analysis (Milestone 5)
8. Deployment & Documentation (Milestone 6)
9. Conclusion and Future Work
10. References and Appendix

4. Recommended Folder Structure

```
project/
├── app/                # Streamlit/Gradio app
├── api/                # FastAPI/Flask service (if any)
├── src/                # Model & pipeline code
├── notebooks/         # Training & evaluation notebooks
├── data/              # Dataset structure (links or samples)
├── models/            # Saved models or weights
├── docs/              # All documentation (below)
│   ├── overview.md
│   ├── technical_doc.md
│   ├── user_guide.md
│   ├── api_doc.md
│   └── licenses.md
├── requirements.txt
└── README.md          # Executive summary + demo links
```

5. Additional Nice to have things:

- Record a short demo video (10-20 minutes) and embed the YouTube or Hugging Face link in the README.
- Add a Quick Start in GitHub Readme.
- Mention resource constraints (GPU, latency, storage) and how they were managed.

If you are wondering what is the difference and level of detail between documentations in 2 and 3 above here is a gist.

Item	Comprehensive Technical Documentation	Final Project Report
Audience	Developers / evaluators	Reviewers / instructors
Tone	Procedural and technical	Analytical and narrative
Goal	Reproducibility & maintainability	Communication & assessment
Location	/docs/ folder, Markdown format	PDF or .docx submission
Includes	Setup, configs, architecture, code details	Summaries, results, interpretation, conclusions